

a) Problem: Different types of Convolution Mode

Logical Solution:

- $y(n) = \sum_{k=-\infty}^{\infty} x(n) * h(n - k)$
- Full $\rightarrow$ all possible combinations,
- Same $\rightarrow$ same length of larger input.
- Valid $\rightarrow$ where all the signal values are engaged for calculation.

Source Code:

```
import numpy as np
import matplotlib.pyplot as plt

x_n=np.array([4,8,9,10])
h_n=np.array([1,1,1,1])
h0=0
y_n=np.convolve(x_n,h_n,'full')
length=len(y_n)
t=[i for i in range(length)]

plt.grid(True)
plt.stem(t,y_n)
plt.show()
```

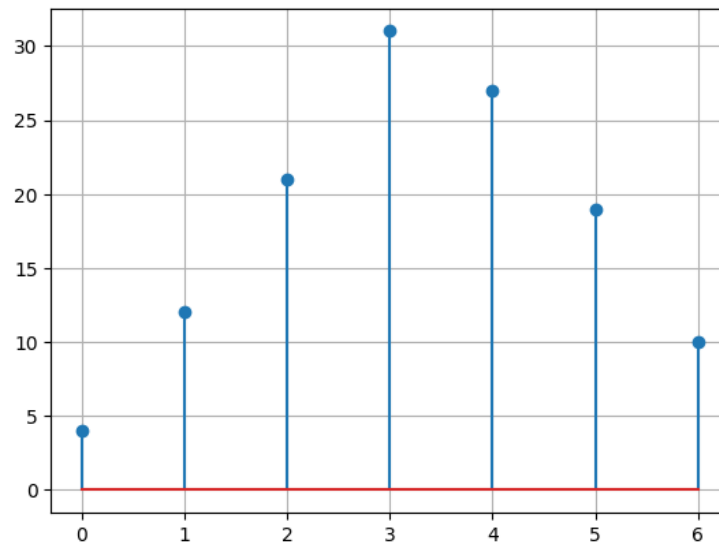


Figure-Full Mode Convolution

```
y_n=np.convolve(x_n,h_n,'same')  
length=len(y_n)  
t=[i for i in range(length)]
```

```
plt.grid(True)  
plt.stem(t,y_n)  
plt.show()
```

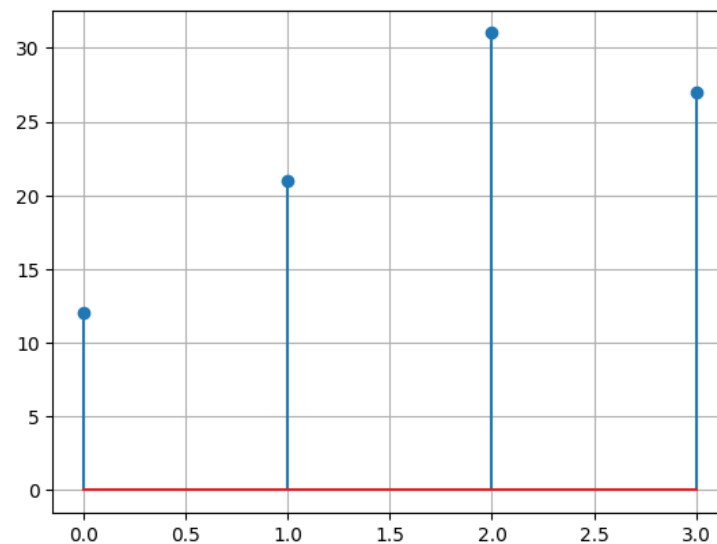


Figure- Same Mode Convolution

```
y_n=np.convolve(x_n,h_n,'valid')  
print(y_n)  
length=len(y_n)  
t=[i for i in range(length)]
```

```
plt.grid(True)  
plt.stem(t,y_n)  
plt.show()
```

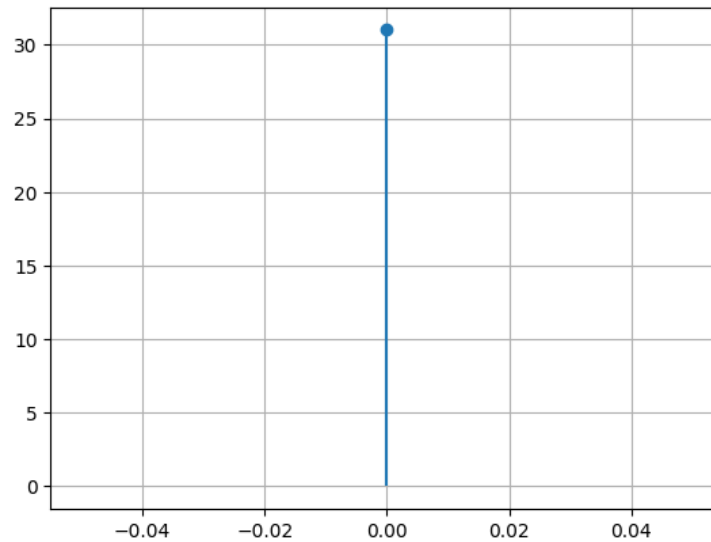


Figure- Valid Mode Convolution

Discussion:

- Using built-in convolution function I have implemented the different kinds of convolution mode.

b) Problem: Edge Detection using Convolution

Logical Solution:

- $$y(n) = \sum_{k=-\infty}^{\infty} x(n) * h(n - k)$$

Source Code:

```
import numpy as np
import matplotlib.pyplot as plt

x0=[0]*15
x1=[1]*15
x_n=x0+x1+x0+x1+x0+x1+x0
kernel=[1,-1]
edges=np.convolve(x_n,kernel)
```

```
fig,(ax1,ax2)=plt.subplots(2,1,figsize=(9, 8))
```

```
ax1.stem(x_n)
```

```
ax1.set_title("Original Signal x[n]")
```

```
ax1.grid(True)
```

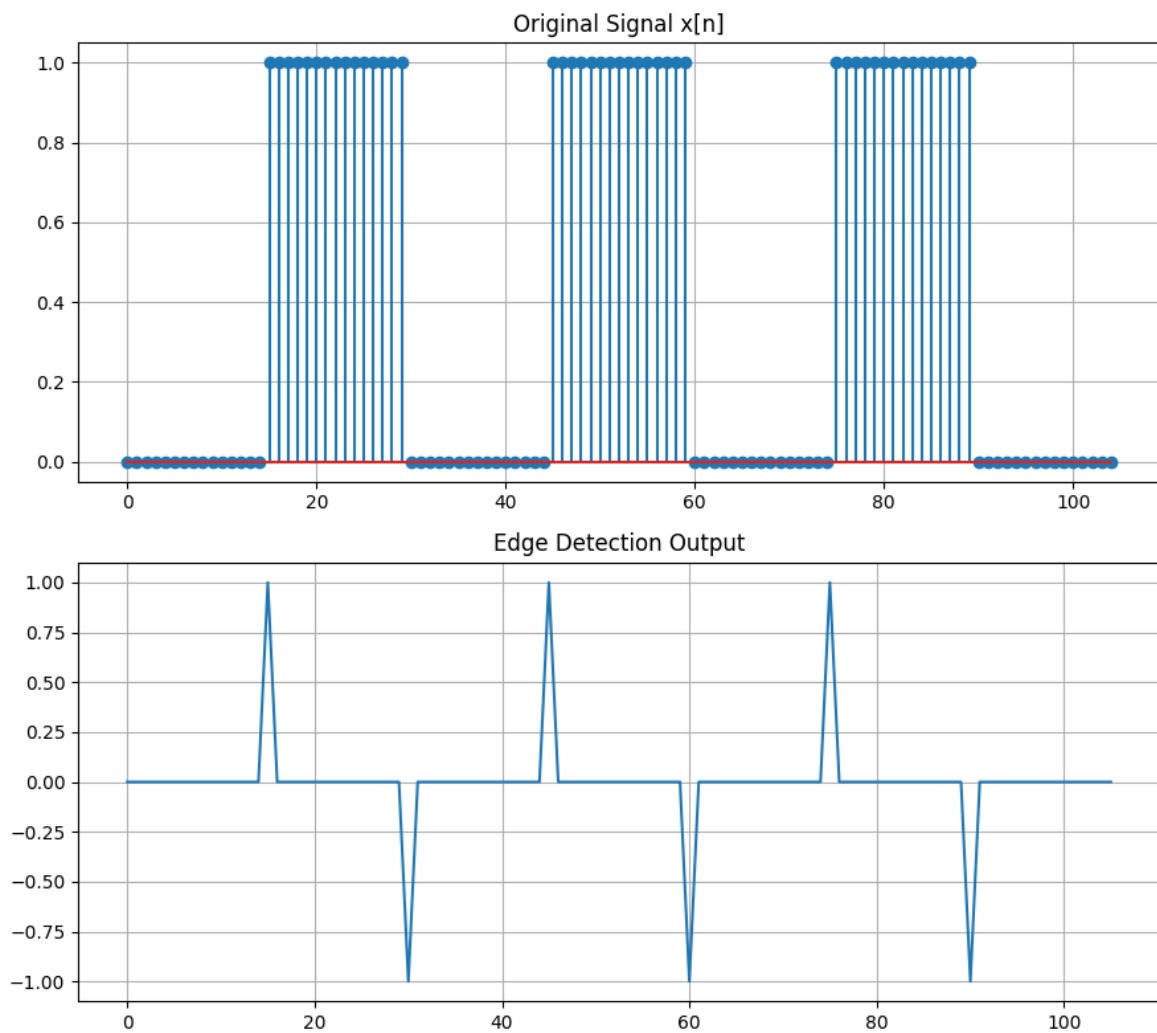
```
ax2.plot(edges)
```

```
ax2.set_title("Edge Detection Output")
```

```
ax2.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```



Output- Edge Detection of sharp Transition

Discussion:

- Using matplotlib library and Built-in numpy function I have figured out the spike of the edge.

c) Problem: Different Kinds of Correlation**Logical Solution:**

- $$r_{xy}(n) = \sum_{k=-\infty}^{\infty} x(k) * h(k - n)$$

Source Code:

```
import numpy as np
import matplotlib.pyplot as plt

def correlation(x,x0,y,y0):
    t=[]
    rxy=[]
    # rxy(n)= sum(k=-inf to inf) x(k)*y(k-n)
    mx=max(len(x),len(y))
    for n in range(-mx,mx+1):
        rxy.append(0)
        t.append(n)
        for k in range(-mx,mx+1):
            try:                # if positive index exist?
                if k>=(-x0): a=x[k+x0] # if negative index exists?
            else: a=0
            except: a=0

            try:
                if (k-n)>=(-y0): b=y[k+y0-n]
            else: b=0
            except: b=0

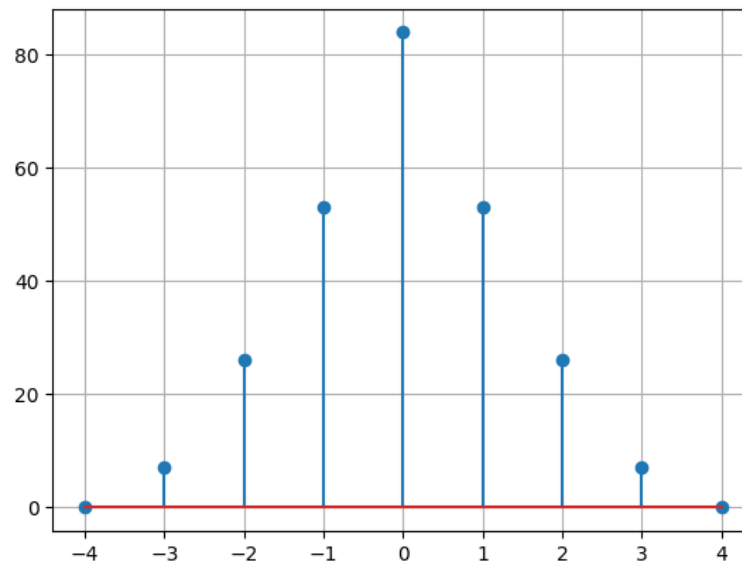
            rxy[-1]+=a*b

    return t,rxy
```

Auto-correlation

```
x=np.array([1,3,5,7])  
x0=0  
t,rxxy=correlation(x,x0,x,x0)
```

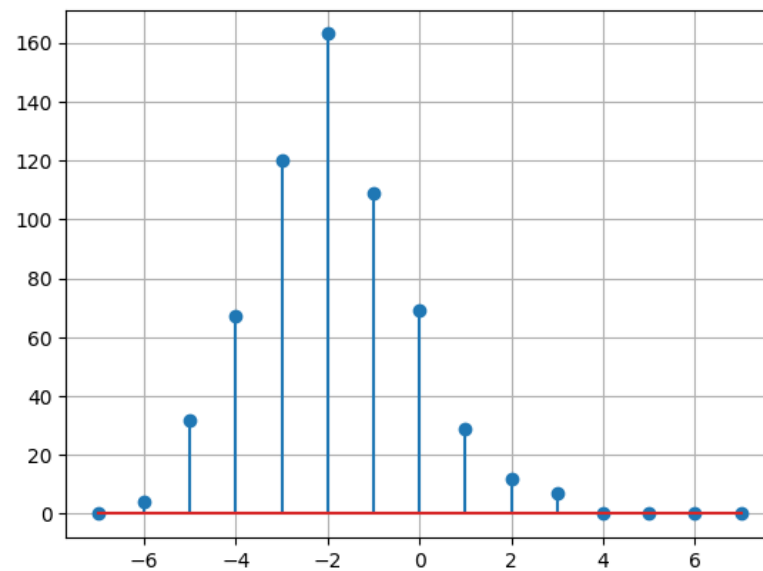
```
plt.stem(t,rxxy)  
plt.grid(True)  
plt.show()
```



Output- Auto-Correlation

cross-correlation:

```
x=np.array([2,3,5,7])  
x0=0  
y=np.array([1,1,3,7,9,13,2])  
y0=0  
t,rxxy=correlation(x,x0,y,y0)  
plt.stem(t,rxxy)  
plt.grid(True)  
plt.show()
```



Output- Cross Correlation

Auto Correlation using built-in function

f=1

t=np.linspace(-1,1,1000)

sinx=np.sin(2*np.pi*f*t)

cosx=np.cos(2*np.pi*f*t)

plt.plot(sinx)

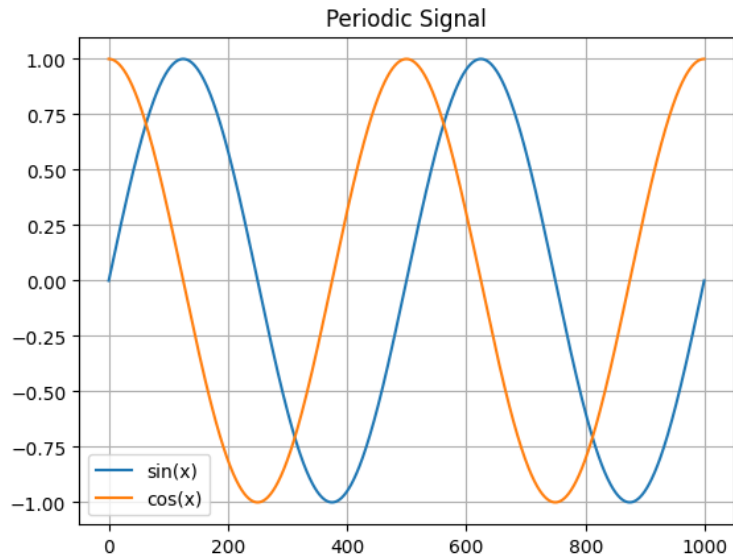
plt.plot(cosx)

plt.grid(True)

plt.legend(('sin(x)', 'cos(x)'))

plt.title("Periodic Signal")

plt.show()



Output- sine-cosine signal

Sine Auto-correlation

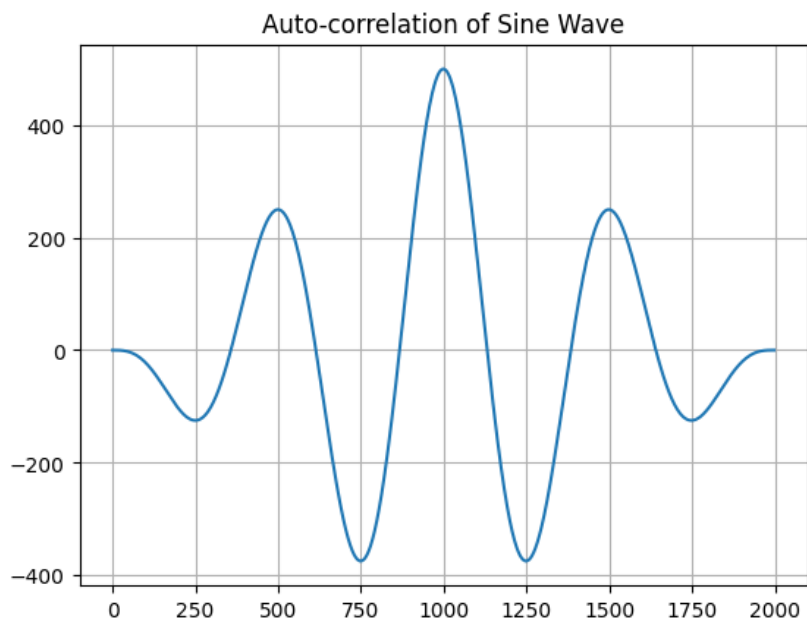
```
auto_correlate=np.correlate(sinx,sinx,mode='full')
```

```
plt.plot(auto_correlate)
```

```
plt.title("Auto-correlation of Sine Wave")
```

```
plt.grid(True)
```

```
plt.show()
```



Output- Auto-correlation of Sine signal

Cosine Auto-correlation

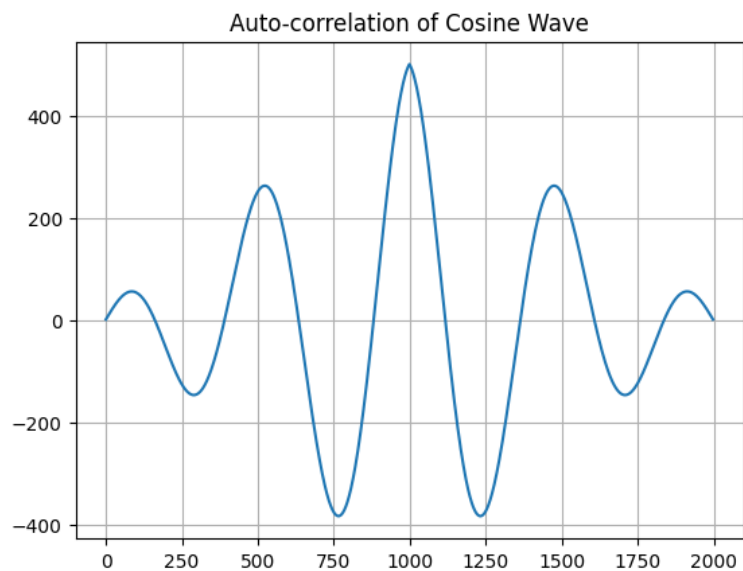
```
auto_correlate=np.correlate(cosx,cosx,mode='full')
```

```
plt.plot(auto_correlate)
```

```
plt.title("Auto-correlation of Cosine Wave")
```

```
plt.grid(True)
```

```
plt.show()
```



Output- Auto-correlation of Cosine signal

Discussion:

- I have implemented the auto and cross correlation for normal signal value and periodic signal value.